

Análise de Código Java

Consistent Hashing — Explicação Técnica em Português

1. Objetivo Geral

A classe **ConsistentHashing** implementa o algoritmo de *Hashing Consistente*, uma técnica usada em sistemas distribuídos para distribuir requisições (ou dados) entre múltiplos servidores de forma equilibrada e resiliente. O objetivo é minimizar a redistribuição de chaves quando nós são adicionados ou removidos do cluster, ao contrário do hashing tradicional que remapeia quase todas as chaves nessas situações.

2. Lógica e Algoritmo

O algoritmo organiza nós e chaves num *anel virtual* de hashes. Para cada servidor real, são criados **N nós virtuais** (definidos por `virtualNodes`), cada um inserido no `TreeMap` com um hash calculado a partir do nome do servidor concatenado com um índice. Para rotear uma chave a um servidor, calcula-se o seu hash e usa-se `ceilingEntry()` para encontrar o nó virtual imediatamente superior (ou o primeiro, caso não exista — simulando o comportamento circular do anel). A remoção de um servidor elimina todos os seus nós virtuais do anel.

```
Map.Entry<Integer, String> entry = ring.ceilingEntry(hash); return (entry != null  
? entry : ring.firstEntry()).getValue();
```

3. Conceitos e Funcionalidades Java

TreeMap<Integer, String>	Mantém as entradas ordenadas por chave (hash), essencial para a busca eficiente com <code>ceilingEntry()</code> .
hashCode()	Função nativa do Java usada para mapear strings a inteiros — serve como função de hash do anel.
Nós Virtuais	Cada servidor é representado por múltiplos pontos no anel, melhorando a distribuição uniforme das chaves.
Map.Entry<K,V>	Interface usada para retornar pares chave-valor do <code>TreeMap</code> nas operações de lookup.
Genéricos (Generics)	Tipagem paramétrica garante segurança de tipos em tempo de compilação na estrutura do anel.

4. Complexidade

Operação	Tempo	Espaço
addNode / removeNode	$O(V \log(N \cdot V))$	$O(N \cdot V)$
getNode	$O(\log(N \cdot V))$	—

Onde **N** = número de servidores e **V** = número de nós virtuais por servidor.

5. Decisões de Design e Limitações

O uso de **nós virtuais** é a principal decisão de design: sem eles, a distribuição de chaves tende a ser desigual. A escolha do `hashCode()` nativo do Java é simples, mas pode gerar colisões ou distribuição não uniforme em cenários de produção — funções como *MurmurHash* ou *SHA-1* são mais adequadas para ambientes reais. Além disso, a implementação não é *thread-safe*: acessos concorrentes ao `TreeMap` sem sincronização podem causar inconsistências. Por fim, o código não persiste o estado do anel, limitando-o a uso em memória dentro de uma única instância JVM.